04-13-2026

Familiar Faces Prototype: Passive Facial Recognition for Automated Name Recall

Hudson Bradley
Oakland University

Abstract

A brief summary of the project's problem, approach, and key results. Remembering names can be a challenging task, especially when in a fast paced environment with lots of people. While some people have a knack for remembering the names of everyone they meet, others can struggle to quickly and correctly recall names. Remembering names isn't just a common courtesy, but also a strategy that can help increase workplace respect and build better relationships with others. While various psychological techniques have been developed to help with name recall, this problem lends itself very well to a technological solution. Familiar faces is the answer to this problem. Familiar faces uses a novel recognition pipeline to passively detect and capture people's faces as the user interacts with them. Then, after the user adds a name to the detected face, a facial recognition algorithm is used to identify a named contact and then project their name temporarily above their head. This prototype iteration demonstrated promising results in both qualitative and quantitative testing. The face detection algorithm showed a high degree of accuracy when trained and tested on the wider-face dataset. These results were further backed by user acceptance tests. Users noticed that familiar faces presented an unobtrusive and helpful reminder when tested in a simulated environment. These early tests show that a technological solution to the name memory problem is not only feasible, but a practical and useful solution.

Introduction

Remembering people's names can be challenging when working in environments where you are meeting lots of people all at once. Professors, conference speakers, and teachers are all examples or roles where knowing someone's name is vital to properly communicating an idea, but where actually remembering everyone's name can be challenging.

The ability to quickly and accurately recall someone's name is not as insignificant as it may seem at first. Studies have shown that there is a perceived spotlight effect that takes place in human's brains that biases them towards overestimating others' perception of them in public settings [1,2]. While the studies show that these fears are largely unfounded, there is still some credence to the person making the mistake that they have just embarrassed themselves by making a public blunder [1,2] Many people can probably relate to the feeling of forgetting a new acquaintance's name and spending the duration of their interaction trying to remember the name, or avoiding needing to say it. It can also be embarrassing to repeatedly ask someone to remind you of their name. While once or twice may be completely normal, continuous name reminders can be annoying or even insulting [3].

Remembering a person's name isn't just important for escaping from social embarrassment, it is also vital to gaining the trust of another and building a personal relationship with them [3]. Studies in request compliance have also shown a direct correlation between increased request compliance and proper name recall [4]. Remembering someone's name isn't just a nice courtesy, it is also a leadership tactic that builds rapport with others, and helps build a cohesive team. For classroom settings, studies have also shown that being acquainted with the names of the students helps to build efficiency and conform among the group, which is vital to a productive learning environment [5].

There are several popular methods for remembering people's names such as "The Name Game," which involves sequentially concatenating the names of previous members until all the names have been strung together [5]. This has proven to be highly effective in recalling people's names, and also for increasing classroom/meeting participation [5]. However, "The Name Game" requires a structured setting in which all participants are present and willing to participate. This approach is not applicable to more casual situations, in which new people are coming and going, and there is no formal structure present. This is where Familiar Faces comes in. Familiar faces (prototype) is an application designed to assist in name recall. Familiar faces passively monitors the people you come into contact with throughout the day and looks for common and prolonged interactions. By learning people's faces, familiar faces can help remind you of a person's name whom you haven't seen for a while, or whom you only recently met. While this approach doesn't completely remove the need for the user to remember a person's name, it helps to supplement that effort by providing gentle reminders.

This report explains how the prototype version was designed and implemented. The prototype version is designed as a functionality implementation, rather than an immediately deployable solution. The results of this prototype will be used to influence future development on familiar face's recognition pipeline, with the final version planned for development using the Android XR framework.

The report is structured as follows: the Data section outlines what data sources were used to train the used models, the Methods section details how the program is structured and designed, the Results and Discussion section explains how the interface works and what the testable model performance is. Finally, the Conclusion section

summarises the program performance and what future iterations will improve on.

Data

The unique aspect of familiar faces is that it builds its face dataset passively as it is used. However, the machine vision model used to identify faces was trained from scratch using the Wider-face dataset [6]. This dataset was used to train a YOLOv26 nano detection model. The original dataset was annotated using the PASCAL VOC format, but the YOLO model uses a custom YOLO format for data annotations. The conversion can be quite complicated, so a pre-converted dataset was found on Kaggle and used to train the YOLO model [7]. The dataset required no preprocessing steps since it had already been prepared by the original authors to be used as a benchmarking dataset. The wider-face dataset contains 32,203 images and 393,703 annotated faces, making it a perfect dataset for training an accurate and reliable detection model.

Methods

The architecture of this program can be divided into two main sections: detection, and categorization. Each of these sections contributes to familiar face's functionality by handling tasks such as face detection and recognition, as well as embedding storage and name display. In the following sections, each of these major components will be explained in depth as well as the overall flow of information used by familiar faces.

Face Detection and Tracking

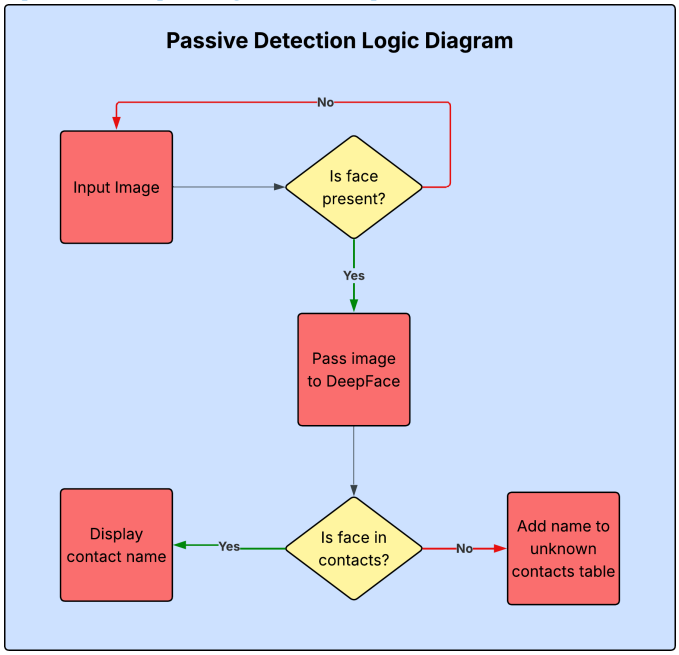
Face detection is a crucial part of familiar face's operation. Familiar faces is designed to run on edge devices in limited compute environments with little to no reliance on cloud computing. As such, it was vital that this program be designed in such a way that it could operate smoothly on a variety of hardware in near real time. While this particular implementation is a prototype, meaning that it is designed to show that familiar faces as a concept can work rather than be the final product, the design constraints of edge hardware were still adhered to.

Face detection was performed by a custom trained YOLOv26 nano object detection model. This is the latest vision model from Ultralitics and is characterized by a shift towards more deployment-ready model architectures [8]. These design choices can be seen in many aspects of YOLOv26's design such as novel optimization techniques, and the removal of computationally complex operations like non-maximum suppression and distributed focal loss [8]. These are just a few of the more notable architectural changes made in YOLOv26 which made it a great choice for face detection in familiar faces.

However, despite these major performance improvements, running YOLO detection on every frame on an edge device would still prove to be too computationally heavy to be practical. So, to compensate for these calculations two important design decisions were made. First, face detection was not run on every frame, but rather every 10 frames. 10 frames was chosen somewhat arbitrarily, but with the intent to show that detection can still work with intermittent face detection. The detection was also not run on the main UI thread, but ran asynchronously in a separate thread that would watch the frame count and only perform detection at a set interval. The results of the detection were then handed to the main UI thread via a queue. The second important design decision was the use of a kalman filter to track faces in between detections. The kalman filter was employed due to its lightweight nature and ability to estimate where a face might be moving to, based on pixel velocities. This allows the face to be tracked in between detections resulting in minimal loss to tracking accuracy. Another benefit provided by the kalman filter was the ability to give each detected face a unique identity. By giving each face its own identity, familiar faces is able to distinguish between separate faces and treat them as separate people, preventing identity blending.

These two elements of the face detection and tracking pipeline were the backbone of the program. They allow faces to be detected and tracked throughout a frame while maintaining separate identities and not overly taxing the hardware. From this solid foundation, the core functionality of familiar faces (e.g. facial recognition) was built as outlined in the following section.

Figure 1. This diagram shows the logic flow (L0 Diagram) of the Face Detection Pipeline. The face tracking step has been omitted as it is part of the "Input Image" collection process.



Facial Recognition

Facial recognition is the fundamental aspect of familiar faces which allows this program to function as a personal name assistant. For this part of the program, a custom model was not used. The reason behind this choice was mainly due to time constraints. While there are a number of resources available to train facial recognition models such they are often very complex and require dedicated GPUs and long training times. For this project, while access to somewhat modern GPUs were available, after training the YOLO model for facial recognition (a process which took ~12hours), it was decided to use a pretrained model for this part of the project.

After some research, it was decided that the Python library DeepFace would be used to implement the FaceNet facial recognition model by Google. DeepFace describes itself as a "lightweight face recognition and facial attribute analysis framework for python" [9]. DeepFace provides functions to access many popular facial recognition models and perform inference using those models. For this application, the FaceNet model by Google was used [10]. FaceNet is a deep convolutional neural network which uses Zeiler&Fergus and Inception style networks to convert images of faces into

high-dimensional vector embeddings [10]. Since FaceNet converts faces into high-dimensional vectors, it was the perfect model for the familiar faces recognition needs.

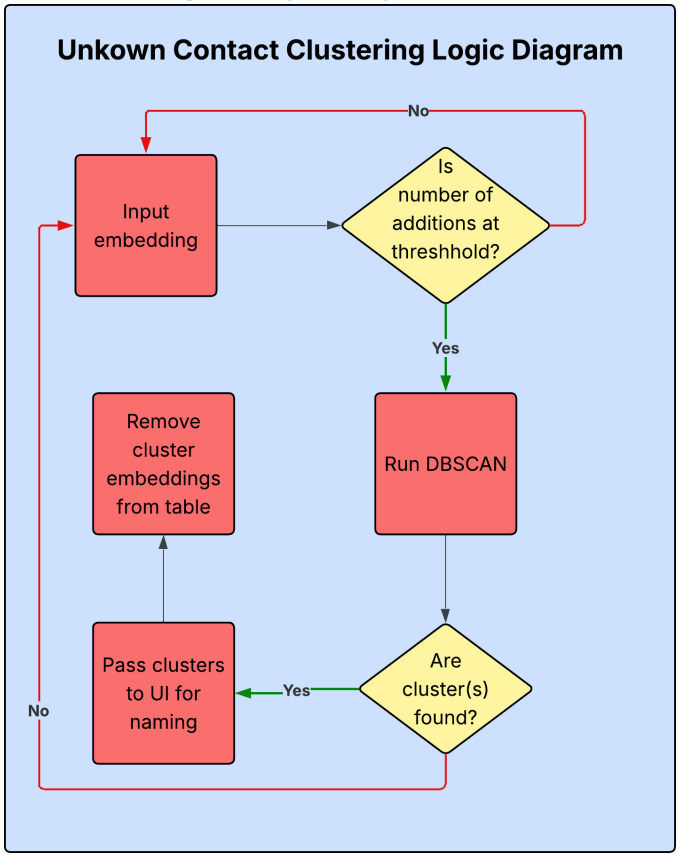
FaceNet sat just downstream of the face detection pipeline. After a face was properly detected, velocity data from the kalman filter was used to watch for when the face had minimal movement to reduce the chance of capturing a blurry face. The face was then cropped out of the frame using the dimensions of the kalman filter. The cropped face was then passed into the DeepFace FaceNet model. Just as with the YOLO detection operation, the FaceNet inference was run asynchronously on a separate thread to prevent this model from causing lag in the main UI thread.

Once the embedding was created, it was normalized using the magnitude of the vector before being passed to a custom KNN function. The KNN function was responsible for determining if the new face was similar to the previously saved faces. If it was, then the name was returned of the person belonging to that face, otherwise the embedding was added to a table in a database for unknown faces.

The KNN algorithm looked at the top five most similar embeddings in the list of known embeddings. If the average distance from these top 5 embeddings was less than a set threshold (0.35) from the new embedding, then the most common name from the list of five names was chosen for the new embedding. Using a KNN algorithm allowed for a better, less-deterministic naming of new embeddings. While KNN is still a deterministic algorithm, by assigning a name from the five most similar known embeddings, familiar faces is able to better handle the faces of people who may appear similar.

Facial Recognition is divided into two parts: facial embedding, and facial comparison. FaceNet is used to embed images of faces into high dimensional vectors which can be used by the KNN algorithm to compare the new face against known faces. This is the core functionality of familiar faces and is what allows the program to tell whether the user knows the name of the person they are looking at, or if they are a new person.

Figure 2. This diagram shows the flow of logic (L0 diagram) for the facial recognition pipeline. The input embedding is generated as a result of the last step in the Figure 1 diagram.



Embedding Storage

The storage of facial embeddings is vital to the usefulness of familiar faces. While storing faces in memory can work, faces need to persist across sessions as well as be managed appropriately so as to keep the embeddings up-to-date. In order to achieve this, familiar faces utilize three tables in an SQLite database. The I/O operations on the database are handled as part of the facial recognition thread described previously. However, the addition of contacts is handled by the main UI thread since that is where the name is collected from the user. To help prevent I/O conflicts by this design choice, each database function is a self-contained I/O operation which opens and closes the database connection independently. The database structure is made up of three tables to handle the necessary information. The three tables are as follows: contacts, embeddings, and unknown_contacts.

Contacts Table

The contacts table was responsible for storing the name, number of encounters, date last seen, and a profile image for each contact the user stores in their contacts list. This table is initially empty and only updates when a new contact is named and added by the user. This table is important for properly storing the name of each contact as well as their contact id which is used by the embeddings table.

Embeddings Table

The embeddings table is where all of the high-dimensional vector embeddings of faces are stored. Each entry stores the id of the contact that the embedding belongs to, the date it was created, and the embedding itself. This table stores all the embeddings for every

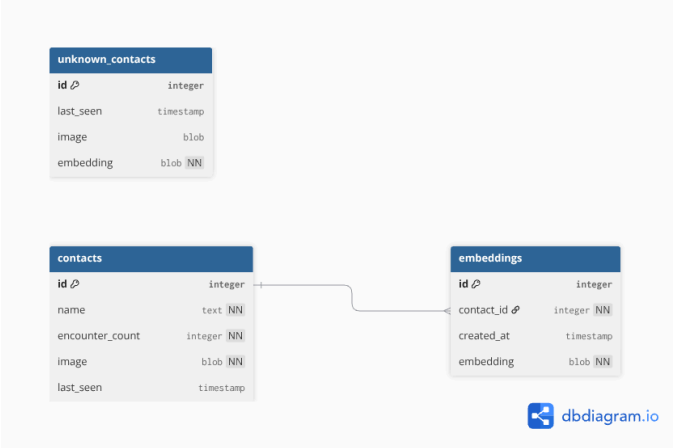
single contact, so it can grow very quickly. In order to limit the size of the database as more contacts are added to the contacts list, each contact has 13 embeddings total. As new embeddings are captured, the oldest embeddings stored in the table are replaced with the newer embedding. This helps to maintain faster search times, while also maintaining recognition accuracy.

Unknown Contacts Table

The unknown contacts table is responsible for storing the date captured, the cropped face image, and the high-dimensional vector of faces which fall outside of the recognition threshold defined in the KNN algorithm. These entries accumulate over time and are used to automatically suggest new contact additions to the user once enough embeddings have been captured.

The use of an SQLite database for storing the contact embeddings, names, dates, and other relevant information is vital to the performance of familiar faces. SQLite operations are quick and come with built-in data safety mechanisms that help prevent two writes from conflicting with each other.

Figure 3. The database diagram that familiar faces uses to store known and unknown contact embeddings.



Unknown Contact Search

The final feature implemented into familiar faces is the unknown contact search algorithm. This algorithm is responsible for finding patterns in the embeddings stored in the unknown_contacts table in the database and presenting these embeddings for naming once a sufficient number have been collected. In order to accomplish this a clustering algorithm is periodically run on the unknown_contacts table and if more than 13 embeddings are found to be similar, they are passed to the main UI thread for naming.

Initially, a K-Means algorithm was thought to be the solution to this clustering problem. However a flaw was quickly discovered, with K-Means a known value for K is required before running the algorithm. The unknown_contacts table is characterized by being unknown, meaning that there is no value of K that can be determined beforehand. To solve this problem, a new density based clustering algorithm was used called DBSCAN.

DBSCAN is a clustering algorithm that is designed to discover clusters of arbitrary shape while requiring minimal input from the user [11]. DBSCAN proved to be a fantastic choice for this program because it is able to find clusters of data without needing any prior knowledge about the data. Using this approach, DBSCAN is able to identify clusters of similar face embeddings automatically. This algorithm was implemented using the implementation found in the Scikit-Learn library. As a part of this implementation, a minimum cluster size and similarity parameter can be set to further fine-tune the clusters.

Using DBSCAN allows familiar faces to passively find similar embeddings of people and present them to the user for naming once a sufficient amount of data has been collected.

Results and Discussion

Since this implementation of familiar faces is intended to be a prototype that tests the usefulness as well as some architectural details that are important for future versions, it is important that both the quantitative as well as the qualitative performance of the system are measured.

In this results and discussion section, the quantitative performance of the facial detection model is reported. Familiar face's usefulness primarily relies on how the user perceives its usefulness, so a brief user acceptance test was conducted and the results are reported and discussed below.

Face Detection Performance

As discussed previously, YOLOv26 nano was the base model used to detect faces in the camera stream. It is important to understand how well the model performs as that insight will aid in explaining the user experience with detection accuracy.

For familiar faces, it is important to understand where the face detection model needs to excel before analyzing the performance metrics. When detecting a face, it is important that the object being passed to the FaceNet model is actually a face. It would be wasteful to both battery life and compute resources if non-face objects were constantly being passed to the FaceNet model by the detection model. Also, it is important that the bounding boxes adequately cover the detected face. Since the dimensions of the bounding box are used to crop the face from the frame, if the boxes are offset or too small or too large, the FaceNet model may fail to properly identify a face, again wasting compute resources.

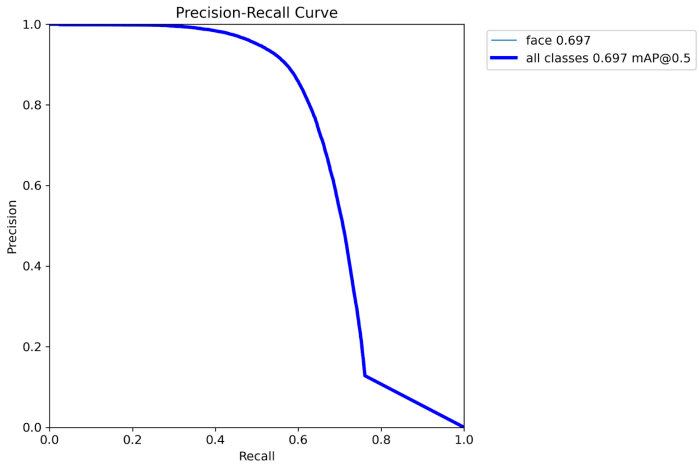
Using these applications of the face detection model to guide the metric evaluation, it makes sense to analyse mAP at various IoU thresholds, precision, and recall. Intersection over Union (IoU) shows how well the model was able to align detection boxes over ground truth. Mean average precision (mAP) is used to show the average precision of the model at various IoU thresholds (0.5-0.95 in this case). These two metrics work in tandem to show how accurate positive predictions were at various degrees of alignment. For these metrics, an ideal value will be a large mAP@0.5-.95 indicating that detection boxes are frequently properly aligned over actual faces. Finally, the precision and recall metrics show how well the model accurately predicts true faces, and how well the model captures all the faces shown.

Table 1. The following metrics are the results of training the YOLOv26 nano model on the Wider-Face dataset for 100 epochs.

Model	**mAP@0.5-0.95**	**Precision**	**Recall**
YOLOv26n	0.37795	0.84665	0.60516

Training metrics show very promising performance results from YOLOv26 nano. The mAP@0.5-0.95 IoU is 37.8% with a mAP@0.5 of 69.7% (Figure 1), meaning that at various IoU thresholds, the model shows a strong balance between precision and recall. For a simple dataset, a mAP@0.5-0.95 of 37.8% may not be great performance; however, due to the highly variable nature of the Wider-Face dataset which has a large variety of faces in differing conditions and sizes, this performance is a strong starting point for a model of this size. Furthermore, the face detection model achieved a precision of 84.67% meaning that of all the bounding boxes predicted by the model, 84.67% contained actual faces. This is great performance for a model of this size and provides more than adequate performance for this application, especially when combined with the mAP performance. Finally, the model obtained a recall of 60.52%. Recall shows how well the model was able to identify all instances of a class. A recall of 60.52% shows that the face detection model was able to detect roughly 60% of all faces in a given image. While this provides a working baseline for the program, it leaves room for improvement in future iterations.

Figure 4. The precision-recall curve of the face detection model (YOLOv26n). Results show a mAP of 0.697 at an IoU of 0.5. This indicates excellent performance given the complex nature of the Wider-Face dataset.



Overall, the face detection model provides fantastic performance given the model size and computational requirements. The model shows a solid mAP at various IoU thresholds meaning that the model accurately finds the bounds of faces in an image. Additionally, the model's precision shows great performance, meaning that when a face is detected, the results show that it is highly likely to be a real face. The recall performance of this model showed good results for the dataset it was tested on. Wider-face is known to be a challenging dataset with multiple faces of varying sizes. So, the face detection model provides more than adequate performance, but when deployed in future mobile iterations there may be need for further training tweaks to increase the model's ability to find all the faces present.

User Acceptance Tests

In order to test how users would perceive the usefulness of an application such as familiar faces, a series of simulated tests were performed with several volunteers to gather feedback on familiar face's performance. The tests were conducted by having participants site down in front of a laptop screen while familiar faces streamed from a forward facing camera. A known contact then approached the camera and the participant and contact had a simulated conversation. Participants were instructed to think out loud as the interaction took place. After the interaction was over, participants were interviewed about the intrusiveness of the pop up, the helpfulness of seeing the contacts name, and finally how distracting the on screen display was.

The majority of participants reported that the name display was unobtrusive and was helpful in knowing the contact's name. This is very important to the acceptance of future familiar face iterations. The name pop-up needs to be unobtrusive and not something that is distracting to the user. It needs to provide relevant information, and then disappear to the user. One participant noted how after reading the name, their focus shifted away from the text causing it to disappear in their vision. This was a positive observance, and is the desired effect for familiar faces. While the display options are limited for this prototype of familiar faces, future iterations will seek to maintain that same effect for the user.

In one of the user tests, the participant was approached from 40 feet away by the contact. The participant noted that familiar faces was able to pick up and correctly identify the contact at that distance and continued to correctly identify the contact as they approached. This was an unexpected but welcome surprise. While this behavior is not expected in every instance, it is a possibility that will need to be designed around in future iterations. While this distance identification is more of a feature than a bug, it is important that the distance from which contacts are identified is limited to be in the range that normal conversations occur. This was very valuable feedback that will help shape future iterations of familiar faces.

In sum, the participants found the concepts of familiar faces very useful and a novel application of machine vision concepts. They expressed how the showing of a person's name was unobtrusive to the conversation while also being helpful for identifying contact names. This provides highly valuable feedback on familiar face's application and execution that will help shape future iterations of this program.

Conclusion

In conclusion, familiar faces is a novel passive facial recognition application designed to help users remember the names of people they've recently met. Familiar faces utilizes a four part detection pipeline which uses YOLOv26n for face detection, DeepFace for face embedding, KNN for facial recognition, and DBSCAN for automatic unknown face identification. This allows familiar faces to passively find faces the user encounters, then cluster them into new contacts automatically which can be used for facial recognition later.

There are a couple of limitations with this pipeline that have been observed during testing. The most prominent issue is that faces in different head positions (looking up/down, or left/right) are sometimes recognized as separate faces. This is most likely a result of two different problems: the KNN recognition algorithm, and the DBSCAN algorithm. The DBSCAN algorithm needs further tuning in order to more accurately cluster the same face in different positions. While it is not suspected that DBSCAN is the wrong clustering algorithm for this application, it is possible that there is another better algorithm for this application. Furthermore, the KNN algorithm, which is responsible for determining whether a face is known or not, is suspected to be the cause of this problem. While tuning the hyperparameters of this algorithm may help reduce the number of false negatives, it is more likely that the algorithm itself

needs to be replaced. Another clustering algorithm such as Approximate Nearest Neighbor (ANN) might provide better consistency in detection.

Familiar faces prototype marks the first iteration of familiar faces. There is a lot of future work that needs to be done to make familiar faces production ready. Future work will entail further tuning of familiar face's performance to find the best passive recognition pipeline. Once a solid recognition pipeline has been found, the next step will be converting this pipeline to a mobile application. From there, familiar faces can be tested further in a more realistic compute environment. The final iteration of the development process will be converting the mobile application to the android XR framework. This step will also include more extensive testing on the wearable hardware and depending on the results, may require another change to the recognition pipeline. So, while this report documents the prototype version of familiar faces, it lays vital groundwork that will be used to influence future iterations of familiar faces.

References

1. Hargis, M.B., Whatley, M.C., and Castel, A.D., "Remembering proper names as a potential exception to the better-than-average effect in younger and older adults.," *Psychol. Aging* 35(4):497–507, 2020, doi:10.1037/pag0000472.

2. Gilovich, T. and Savitsky, K., "The Spotlight Effect and the Illusion of Transparency: Egocentric Assessments of How We Are Seen by Others," *Curr. Dir. Psychol. Sci.* 8(6):165–168, 1999, doi:10.1111/1467-8721.00039.

3. Social Skills Center, "Why Remembering Names Leads to Greater Social Connections," https://socialskillscenter.com/why-remembering-names-leads-to-greater-social-connections/, n/a.

4. Howard, D.J., Gengler, C., and Jain, A., "What's in a Name? A Complimentary Means of Persuasion," *J. Consum. Res.* 22(2):200–211, 1995.

5. Morris, P.E. and Fritz, C.O., "The name game: Using retrieval practice to improve the learning of names.," *J. Exp. Psychol. Appl.* 6(2):124–129, 2000, doi:10.1037/1076-898X.6.2.124.

6. Yang, S., Luo, P., Loy, C.C., and Tang, X., WIDER FACE: A Face Detection Benchmark, 2015, doi:10.48550/arXiv.1511.06523.

7. CanOmercik, mustafayngl, "WIDER Face Dataset for YOLO12," https://www.kaggle.com/datasets/canomercik/wider-face-dataset-for-yolov12-format, n/a.

8. Sapkota, R., Cheppally, R.H., Sharda, A., and Karkee, M., YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection, 2025, doi:10.48550/ARXIV.2509.25164.

9. Serengil, S.I. and Ozpinar, A., Boosted LightFace: A Hybrid DNN and GBM Model for Boosted Facial Recognition, *Gazi Univ. J. Sci.* 39(1):452–466, 2026, doi:10.35378/gujs.1794891.

10. Schroff, F., Kalenichenko, D., and Philbin, J., "FaceNet: A Unified Embedding for Face Recognition and Clustering," 2015, doi:10.48550/ARXIV.1503.03832.

11. Ester, M., Kriegel, H.-P., Sander, J., and Xu, X., "A density-based algorithm for discovering clusters in large spatial databases with noise," *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, AAAI Press, Portland, Oregon: 226–231, 1996.